

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

СибГУТИ

РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА

Тема: Разработка сетевого приложения. Чат

Мультипроцессная реализация сервера с установлением
соединения с использованием функции fork

Выполнил:

студент группы ИП-312

Прозоренко Константин Владиславович

Проверил:

Преподаватель

Ревун Артем Леонидович

Новосибирск, 2026

Содержание

1	Постановка задачи	3
2	Описание протокола	4
2.1	Основные характеристики TCP	4
2.2	Установление и завершение соединения	4
2.3	Порты и адресация	5
2.4	Программный интерфейс сокетов Беркли	5
2.5	Фрагментация данных в TCP	5
3	Описание реализации	6
3.1	Запрос (request)	6
3.2	Ответ (response)	7
3.3	Уведомление (notification)	7
3.4	Аутентификация	7
3.5	Маршруты API	9
3.6	Архитектура сервера	10
3.7	Обработка клиента	10
3.8	Маршрутизация запросов	11
3.9	Система уведомлений	12
3.10	Хранение данных	12
4	Скан экрана работы программы	14
5	Список источников	16

1 Постановка задачи

Необходимо разработать сетевое приложение «Чат», реализующее обмен текстовыми сообщениями между пользователями в реальном времени.

Серверная часть должна быть реализована с использованием мультипроцессной архитектуры: для каждого входящего соединения создается отдельный процесс с помощью системного вызова `fork()`.

Требования к серверной части:

- Регистрация и аутентификация пользователей по логину и паролю.
- Обмен личными сообщениями между двумя пользователями.
- Создание групповых чатов, добавление и удаление участников.
- Отправка уведомлений клиентам в реальном времени.
- Хранение данных в файлах формата JSON с блокировками для конкурентного доступа.
- Мультипроцессная обработка соединений с помощью `fork()`.

2 Описание протокола

Для передачи данных между клиентом и сервером используется протокол TCP (Transmission Control Protocol) — один из основных протоколов транспортного уровня стека TCP/IP. TCP обеспечивает надежную, ориентированную на соединение доставку потока байтов между процессами, работающими на разных хостах в сети.

2.1 Основные характеристики TCP

- **Ориентация на соединение.** Перед началом передачи данных устанавливается логическое соединение между отправителем и получателем. Соединение однозначно идентифицируется парой сокетов: (IP-адрес отправителя, порт отправителя, IP-адрес получателя, порт получателя).
- **Надежная доставка.** TCP гарантирует, что все отправленные данные будут доставлены получателю без ошибок, потерь, дублирования и с сохранением порядка следования. Для этого используются механизмы подтверждений (ACK), порядковых номеров сегментов и повторной передачи при таймаутах.
- **Потоковая модель.** TCP передает непрерывный поток байтов, не сохраняя границы отдельных сообщений прикладного уровня.
- **Дуплексная связь.** Данные могут передаваться одновременно в обоих направлениях.
- **Управление потоком (flow control).** TCP использует скользящее окно для предотвращения переполнения буфера получателя.
- **Управление перегрузкой (congestion control).** TCP регулирует интенсивность отправки данных в зависимости от загрузки сети.

2.2 Установление и завершение соединения

Установка соединения TCP выполняется по процедуре «тройного рукопожатия» (three-way handshake):

1. Клиент отправляет серверу сегмент с флагом SYN, содержащий начальный порядковый номер клиента ISN_c .
2. Сервер в ответ отправляет сегмент с флагами SYN+ACK, содержащий свой начальный порядковый номер ISN_s и подтверждение номера клиента $ISN_c + 1$.
3. Клиент отправляет сегмент с флагом ACK, подтверждающий получение $ISN_s + 1$.

После этого соединение считается установленным, и начинается передача данных.

Завершение соединения выполняется аналогичной процедурой с использованием флага FIN. Каждая сторона может завершить свою половину дуплексного соединения независимо.

2.3 Порты и адресация

TCP использует 16-битные номера портов для адресации конкретного приложения на хосте. Порт сервера фиксирован (в нашем приложении — 8080 по умолчанию). Порт клиента назначается операционной системой автоматически из диапазона эфемерных портов (обычно 32768–60999).

2.4 Программный интерфейс сокетов Беркли

Взаимодействие с TCP в UNIX-подобных операционных системах осуществляется через API сокетов Беркли (Berkeley Sockets). Основные системные вызовы:

- `socket(AF_INET, SOCK_STREAM, 0)` — создает TCP-сокет. `AF_INET` указывает на использование протокола IPv4, `SOCK_STREAM` соответствует TCP.
- `bind()` — привязывает сокет сервера к конкретному адресу и порту.
- `listen()` — переводит серверный сокет в режим прослушивания, задает очередь ожидающих соединений (`backlog`).
- `accept()` — принимает входящее соединение, возвращая новый сокет для обмена данными с клиентом.
- `connect()` — устанавливает соединение с сервером (со стороны клиента).
- `read()` / `write()` (или `recv()` / `send()`) — чтение и запись данных в сокет. TCP рассматривает данные как поток, не сохраняя границы сообщений.
- `close()` — завершает соединение и освобождает ресурсы сокета.

2.5 Фрагментация данных в TCP

Поскольку TCP передает поток байтов без сохранения границ сообщений, приложение должно самостоятельно реализовать механизм кадрирования (`framing`). В нашем случае каждое сообщение прикладного уровня завершается символом `\n`, что позволяет получателю однозначно выделять отдельные сообщения из потока.

3 Описание реализации

Поверх TCP реализован собственный прикладной протокол обмена сообщениями в текстовом формате JSON. Сервер принимает соединения на заданном порту (по умолчанию 8080). Каждое сообщение передается как одна строка в формате JSON, завершенная символом перевода строки `\n`. Максимальный размер сообщения — 8192 байта.

Все сообщения имеют поле `kind`, определяющее их тип. Поддерживаются три типа сообщений:

- **request** — запрос от клиента к серверу;
- **response** — ответ сервера на запрос;
- **notification** — асинхронное уведомление от сервера клиенту.

3.1 Запрос (request)

Формат запроса:

```
{
  "kind": "request",
  "route": "/путь/к/ресурсу",
  "type": "GET|POST|DELETE",
  "headers": {
    "Authorization": "токен"
  },
  "content": { ... }
}
```

Поле `route` задает путь к ресурсу в стиле REST API. Поле `type` определяет HTTP-подобный метод. Поле `headers` содержит заголовок `Authorization` с токеном аутентификации (кроме маршрутов регистрации и входа). Поле `content` содержит тело запроса в формате JSON.

3.2 Ответ (response)

Формат ответа:

```
{
  "kind": "response",
  "code": 0,
  "content": { ... }
}
```

Поле code содержит числовой код ошибки:

- 0 — успех (ERR_OK);
- 1 — неверный запрос (ERR_BAD_REQUEST);
- 2 — неавторизован (ERR_UNAUTHORIZED);
- 3 — не найдено (ERR_NOT_FOUND);
- 4 — запрещено (ERR_FORBIDDEN);
- 5 — конфликт (ERR_CONFLICT);
- 6 — внутренняя ошибка (ERR_INTERNAL).

3.3 Уведомление (notification)

Формат уведомления:

```
{
  "kind": "notification",
  "code": 1,
  "content": { ... }
}
```

Поле code содержит тип уведомления:

- 1 — новое сообщение (NOTIF_NEW_MESSAGE);
- 5 — изменение состава участников чата (NOTIF_CHAT_MEMBERS_CHANGED).

3.4 Аутентификация

Токен формируется по схеме login:timestamp:signature, где:

- login — имя пользователя;
- timestamp — Unix-время выдачи токена (time(NULL));
- signature — 16-значное шестнадцатеричное представление хеша djb2 от строки login:timestamp:SECRET_KEY.

Алгоритм djb2 вычисляется по формуле:

hash = 5381

for each byte c: hash = hash * 33 + c

При валидации токена сервер заново вычисляет подпись по извлеченным `login` и `timestamp` и сравнивает с подписью из токена. Токен не имеет срока действия — проверяется только корректность подписи.

Пароли хранятся в виде 16-символьного шестнадцатеричного хеша `djb2` (`%016lx`). Открытый пароль нигде не сохраняется.

3.5 Маршруты API

Аутентификация:

- POST /register — регистрация нового пользователя. Тело: {"login": "...", "password": "..."}.
- POST /login — вход в систему. Тело: {"login": "...", "password": "..."}.
Ответ содержит токен.
- GET /info — информация о сервере (количество пользователей, чатов).

Пользователи:

- GET /users — список всех пользователей.
- GET /users/{login}/messages — личная переписка с пользователем.
- POST /users/{login}/messages — отправка личного сообщения. Тело: {"text": "..."}.

Чаты:

- GET /chats — список чатов текущего пользователя.
- POST /chats — создание чата. Тело: {"name": "..."}.
- DELETE /chats/{name} — удаление чата (только создателем).
- GET /chats/{name}/host — получение создателя чата.
- GET /chats/{name}/users — список участников чата.
- POST /chats/{name}/users — добавление участника. Тело: {"login": "..."}.
- DELETE /chats/{name}/users — удаление участника (или выход из чата).
- GET /chats/{name}/messages — сообщения чата.
- POST /chats/{name}/messages — отправка сообщения в чат. Тело: {"text": "..."}.

3.6 Архитектура сервера

Сервер реализован на языке C с использованием мультипроцессной модели. При инициализации серверный сокет создается с опцией `SO_REUSEADDR` (предотвращает ошибку «Address already in use» при перезапуске сервера), привязывается к `INADDR_ANY` на заданном порту (по умолчанию 8080) и переводится в режим прослушивания с размером очереди `BACKLOG = 16`.

До старта главного цикла выполняется:

- `notify_init()` — выделение разделяемой памяти через `mmap(MAP_SHARED | MAP_ANONYMOUS)` и инициализация межпроцессного мьютекса с атрибутом `PTHREAD_PROCESS_SHARED`;
- регистрация обработчика `SIGCHLD` с флагом `SA_RESTART` — при завершении любого дочернего процесса вызывается `waitpid(-1, NULL, WNOHANG)` в цикле до исчерпания всех завершившихся потомков; для каждого из них вызывается `notify_client_remove()`.

Главный цикл сервера на каждой итерации:

1. Вызывает `notify_dispatch()` — проверяет каналы дочерних процессов через `select()` с нулевым таймаутом и доставляет уведомления клиентам.
2. Вызывает `select()` на серверном сокете с таймаутом 10 мс для ожидания входящего соединения.
3. При поступлении соединения создает неименованный канал (`pipe()`): родитель читает из `pipefd[0]`, дочерний процесс пишет в `pipefd[1]`.
4. Вызывает `fork()`:
 - **Дочерний процесс:** закрывает серверный сокет и читающий конец канала, вызывает `notify_child_init(pipefd[1])` и `handle_client(client_fd)`, после чего завершается.
 - **Родительский процесс:** дублирует `client_fd` через `dup()` (чтобы иметь независимый дескриптор для отправки уведомлений), закрывает `pipefd[1]` и оригинальный `client_fd`, регистрирует запись о клиенте через `notify_client_add(pid, pipefd[0], parent_client_fd)`.

Общая память `SharedClients` хранит счётчик активных клиентов и массив записей `ClientEntry` (до `MAX_CLIENTS`). Каждая запись содержит PID дочернего процесса, его логин, клиентский сокет (у родителя) и читающий конец канала. Доступ к массиву защищён межпроцессным мьютексом.

3.7 Обработка клиента

Дочерний процесс выполняет функцию `handle_client(socket_fd)`. Внутри бесконечного цикла:

1. `read_message()` блокирующе читает из сокета одну строку (до `\n`) в буфер размером `MSG_BUFFER_SIZE = 8192` байт. При разрыве соединения (возврат ненулевого кода) цикл завершается.
2. `parse_request()` разбирает JSON-строку в структуру `Request`. При ошибке парсинга клиенту немедленно отправляется ответ `ERR_BAD_REQUEST`.
3. Если запрос содержит непустое поле `token`, вызывается `validate_token()`. При успехе извлеченный логин сохраняется в локальной переменной `login` и вызывается `notify_register(login)` — логин записывается в запись текущего процесса в разделяемой памяти, что позволяет родителю доставлять уведомления по имени пользователя.
4. Функция `dispatch()` выбирает обработчик по маршруту и HTTP-методу, формирует `Response`.
5. Результат запроса и код ответа выводятся в `stdout` в формате `[login] METHOD /route -> code`.
6. `send_response()` сериализует ответ в JSON и отправляет клиенту через сокет.
7. Память запроса и ответа освобождается (`free_request, cJSON_Delete`).

После выхода из цикла сокет закрывается и дочерний процесс завершается с кодом 0.

3.8 Маршрутизация запросов

Функция `dispatch()` разбирает маршрут с помощью двух вспомогательных функций:

- `route_depth()` — подсчитывает количество сегментов, разделённых символом `/` (например, `/chats/general/messages` → глубина 3);
- `route_segment(route, idx, out, out_size)` — извлекает сегмент по нулевому индексу в буфер `out` размером `MAX_ROUTE_LEN`.

Перед диспетчеризацией извлекаются сегменты `seg0–seg3`. Маршруты проверяются последовательно по комбинации (`seg0, depth, method`). Сегменты с переменными значениями (логин пользователя, имя чата) принимаются из соответствующих позиций (`seg1`).

Маршруты, не требующие аутентификации (`GET /info, POST /register, POST /login`), проверяются первыми — до проверки токена. Все остальные маршруты при отсутствии валидного логина возвращают `ERR_UNAUTHORIZED`. Если ни один маршрут не совпал — возвращается `ERR_NOT_FOUND`.

3.9 Система уведомлений

Уведомления передаются от дочерних процессов родительскому через анонимные каналы (pipe). Каждое уведомление состоит из двух частей:

1. **Заголовок** фиксированного размера `PIPE_HEADER_SIZE = MAX_LOGIN_LEN + sizeof(uint32_t)`: первые `MAX_LOGIN_LEN` байт — логин получателя (строка с нулевым завершителем), затем 4 байта — длина JSON-тела в формате `uint32_t`.
2. **Тело** — JSON-строка уведомления длиной, указанной в заголовке.

Дочерний процесс вызывает `notify_user(login, notif)`: сериализует уведомление через `cJSON_PrintUnformatted()`, формирует заголовок и записывает заголовок+тело в свой `pipefd[1]` двумя последовательными вызовами `write()`.

Для уведомления всех участников чата `notify_chat(chat_name, notif)` получает список логинов из репозитория и вызывает `notify_user()` для каждого.

Родительский процесс в `notify_dispatch()`:

1. Под мьютексом строит `fd_set` из всех зарегистрированных `pipe_read_fd`.
2. Вызывает `select()` с нулевым таймаутом (неблокирующая проверка).
3. Для каждого готового дескриптора вызывает `dispatch_pipe()`: читает заголовок, выделяет буфер, читает JSON, находит `client_fd` нужного пользователя по логину в разделяемой памяти и отправляет уведомление через `write_message()`.

Уведомления отправляются в следующих случаях:

- Новое сообщение в чате (`NOTIF_NEW_MESSAGE`) — уведомляются все участники чата; тело содержит поля `login`, `text`, `chat`, `timestamp`.
- Новое личное сообщение (`NOTIF_NEW_MESSAGE`) — уведомляются отправитель и получатель; тело содержит `login`, `text`, `to`, `timestamp`.
- Изменение состава участников чата (`NOTIF_CHAT_MEMBERS_CHANGED`) — уведомляются все оставшиеся участники; тело содержит `chat`.

3.10 Хранение данных

Данные хранятся в файлах JSON в директории `data/`:

- `data/users.json` — массив объектов `{"login": "...", "password_hash": "..."}.`
- `data/chats.json` — массив объектов чатов с полями имени, создателя и списка участников.
- `data/messages/chat/{chat}.json` — массив сообщений группового чата.

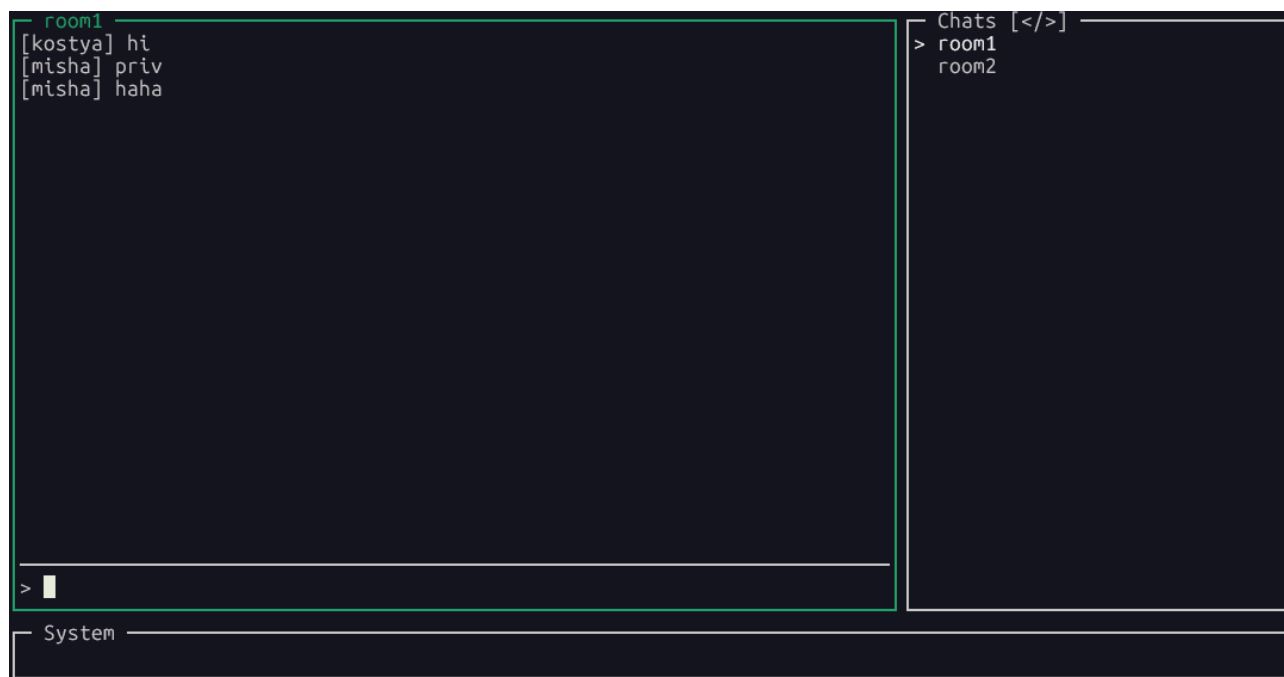
- `data/messages/private/{user1}-{user2}.json` — массив личных сообщений (логины в алфавитном порядке).

Для конкурентного доступа из нескольких процессов используется блокировка файлов через `flock()`. Реализованы два паттерна доступа:

Чтение (`load_json_file`): файл открывается через `open(O_RDONLY)`, затем `flock(LOCK_SH)`. После прочтения и парсинга файловый дескриптор закрывается — блокировка снимается автоматически.

Запись (`locked_load + locked_save`): файл открывается через `open(O_RDWR | O_CREAT, 0644)`, затем `flock(LOCK_EX)`. Дескриптор передается в `locked_save`, который обнуляет файл (`ftruncate`), перемещает позицию в начало (`lseek`), записывает новый JSON и закрывает дескриптор — блокировка снимается. Таким образом, эксклюзивная блокировка удерживается на всё время чтения, изменения и записи данных, что гарантирует атомарность операции обновления.

4 Скан экрана работы программы



PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
11444	diser	20	0	2760	1664	1664	S	0.0	0.0	0:01.39	./bin/server
11787	diser	20	0	2760	1152	1152	S	0.0	0.0	0:00.00	./bin/server
12667	diser	20	0	2760	1152	1152	S	0.0	0.0	0:00.00	./bin/server
11767	diser	20	0	148M	2944	2176	S	0.0	0.0	0:00.02	./bin/client
11786	diser	20	0	148M	2944	2176	S	0.0	0.0	0:00.00	./bin/client
12647	diser	20	0	148M	3072	2304	S	0.0	0.0	0:00.02	./bin/client
12666	diser	20	0	148M	3072	2304	S	0.0	0.0	0:00.00	./bin/client

```
client connected: 127.0.0.1
[anonymous] POST /register -> 0
[anonymous] POST /login -> 0
[misha] GET /chats -> 0
[misha] GET /users -> 0
[misha] GET /users -> 0
[misha] GET /chats -> 0
[misha] GET /chats -> 0
[misha] GET /users -> 0
[misha] GET /chats -> 0
[kostya] GET /chats/room1/messages -> 0
[kostya] POST /chats/room1/messages -> 0
[notify] -> kostya (fd=7)
[kostya] POST /chats/room1/users -> 0
[notify] -> kostya (fd=7)
[notify] -> misha (fd=9)
[kostya] GET /chats/room1/users -> 0
[misha] GET /users -> 0
[misha] GET /chats -> 0
[misha] GET /chats/room1/messages -> 0
[misha] POST /chats/room1/messages -> 0
[notify] -> kostya (fd=7)
[notify] -> misha (fd=9)
[kostya] GET /users -> 0
[misha] POST /chats/room1/messages -> 0
[notify] -> kostya (fd=7)
[notify] -> misha (fd=9)
[misha] POST /chats -> 0
[misha] GET /chats -> 0
```

5 Список источников

1. Stevens W. R., Rago S. A. Advanced Programming in the UNIX Environment. — 3rd ed. — Addison-Wesley, 2013. — 1024 p.
2. Kerrisk M. The Linux Programming Interface. — No Starch Press, 2010. — 1552 p.
3. Stevens W. R., Fenner B., Rudoff A. M. UNIX Network Programming. Volume 1: The Sockets Networking API. — 3rd ed. — Addison-Wesley, 2003. — 992 p.
4. The Open Group. POSIX.1-2017 (IEEE Std 1003.1-2017). — URL: <https://pubs.opengroup.org/onlinepubs/9699919799/> (дата обращения: 29.04.2026).
5. Dave Gamble. cJSON — Ultralightweight JSON parser in ANSI C. — URL: <https://github.com/DaveGamble/cJSON> (дата обращения: 29.04.2026).
6. Bernstein D. J. Hash function djb2. — URL: <http://www.cse.yorku.ca/~oz/hash.html> (дата обращения: 29.04.2026).